

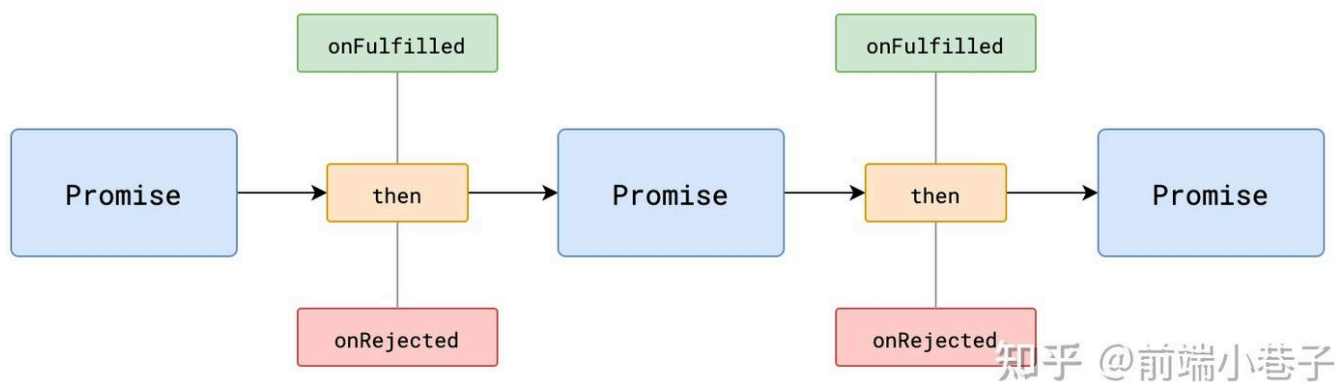
- JS1-Promise-2
 - 2. 链式调用
 - 2.1 原理
 - 2.2 任务链式处理

JS1-Promise-2

2. 链式调用

2.1 原理

Promise 的链式调用是基于 `.then()` 方法的返回值特性实现的。`.then()` 方法总是返回一个新的 Promise 对象，这使得我们可以将多个 `.then()` 方法串联起来，形成链式调用。链式调用的核心在于，每个 `.then()` 方法的返回值都会成为下一个 `.then()` 方法的输入值。



1. 返回值为非 Promise 对象：如果 `.then()` 方法的回调函数返回的是一个非 Promise 对象（如字符串、数字、对象等），那么这个返回值会被包装成一个状态为 `fulfilled` 的 Promise 对象，传递给下一个 `.then()` 方法。

```
Promise.resolve(1)
  .then((value) => {
    console.log(value); // 1
    return 2; // 返回非 Promise 对象
  })
  .then((value) => {
    console.log(value); // 2
  });
```

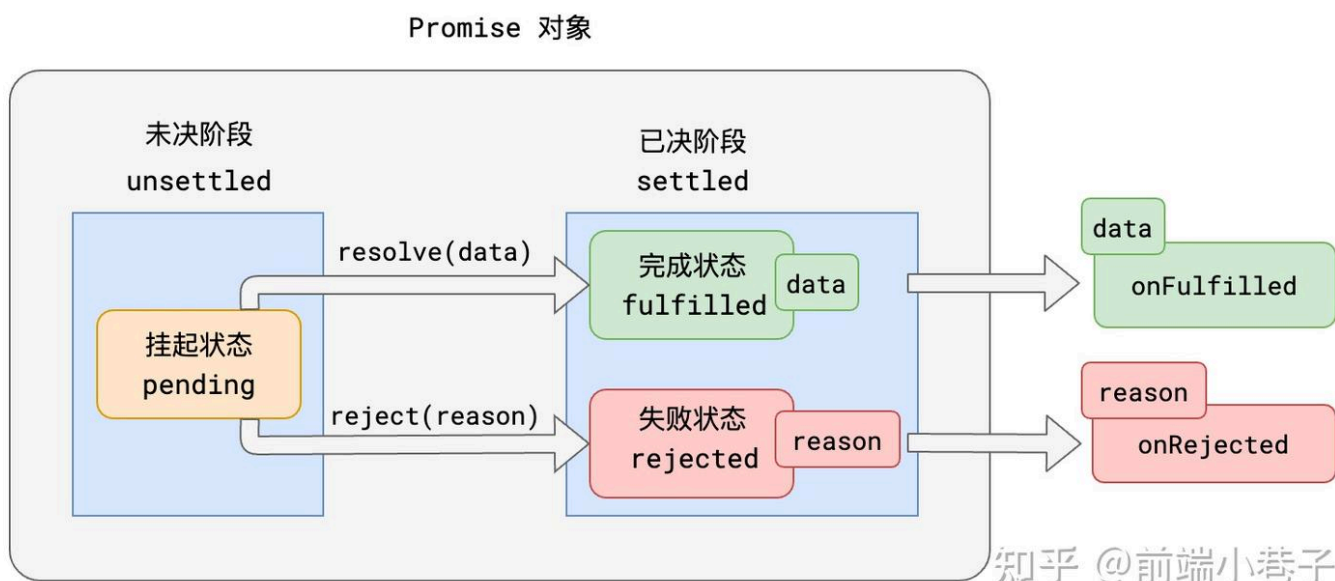
2.返回值为 Promise 对象：如果 `.then()` 方法的回调函数返回的是一个 Promise 对象，那么这个 Promise 对象的状态和值会直接传递给下一个 `.then()` 方法。

```
Promise.resolve(1)
  .then((value) => {
    console.log(value); // 1
    return Promise.resolve(2); // 返回 Promise 对象
  })
  .then((value) => {
    console.log(value); // 2
  });
```

3.抛出异常：如果 `.then()` 方法的回调函数抛出异常，那么这个异常会被捕获，并且链式调用会中断，转而执行 `.catch()` 方法。

```
Promise.resolve(1)
  .then((value) => {
    console.log(value); // 1
    throw new Error('出错了'); // 抛出异常
  })
  .then((value) => {
    console.log(value); // 不会执行
  })
  .catch((error) => {
    console.log(error.message); // "出错了"
  });
```

2.2 任务链式处理



```
function task1() {
  return new Promise((resolve, reject) => {
    console.log('任务1开始');
    setTimeout(() => {
      resolve('任务1完成');
    }, 1000);
  });
}

function task2() {
  return new Promise((resolve, reject) => {
    console.log('任务2开始');
    setTimeout(() => {
      resolve('任务2完成');
    }, 1000);
  });
}

function task3() {
  return new Promise((resolve, reject) => {
    console.log('任务3开始');
    setTimeout(() => {
      resolve('任务3完成');
    }, 1000);
  });
}

task1()
  .then((result) => {
    console.log(result); // "任务1完成"
    return task2();
  })
  .then((result) => {
    console.log(result); // "任务2完成"
    return task3();
  })
  .then((result) => {
    console.log(result); // "任务3完成"
  })
  .catch((error) => {
    console.log(error);
  });
```

在这个例子中，`task1`、`task2` 和 `task3` 是三个异步任务，它们依次执行。每个任务的返回值都会被传递到下一个任务中，最终所有任务都成功完成。

Promise 的链式调用还支持错误处理。如果某个任务失败，可以通过 `.catch()` 方法捕获错误，并进行处理。

```
function task1() {
  return new Promise((resolve, reject) => {
    console.log('任务1开始');
    setTimeout(() => {
      resolve('任务1完成');
    }, 1000);
  });
}

function task2() {
  return new Promise((resolve, reject) => {
    console.log('任务2开始');
    setTimeout(() => {
      reject(new Error('任务2失败'));
    }, 1000);
  });
}

function task3() {
  return new Promise((resolve, reject) => {
    console.log('任务3开始');
    setTimeout(() => {
      resolve('任务3完成');
    }, 1000);
  });
}

task1()
  .then((result) => {
    console.log(result); // "任务1完成"
    return task2();
  })
  .then((result) => {
    console.log(result); // 不会执行
    return task3();
  })
  .then((result) => {
    console.log(result); // 不会执行
  })
  .catch((error) => {
    console.log(error.message); // "任务2失败"
  });
```

在这个例子中，`task2` 抛出了一个错误，通过 `.catch()` 方法捕获了这个错误，并打印了错误信息。